

need to run, or you use them as best you can and fork your code to accommodate different consumption mediums.

With a headless system, you gain complete control over how the content stored in it will be used; and you can design a pipeline that doesn't need to be refactored any time you want to support a new medium for your users, be it a smart watch or an AR system.

If you have run into the challenges mentioned with a traditional CMS, you are most likely using one of the workarounds mentioned as well. If not, you need to start thinking ahead about when you will run into them and how you will address them, starting today.

Leveraging a headless CMS

Headless CMSs not only can be used in places where a CMS is needed but also in other scenarios where you need to store content in a structured and consistent format. Let's explore this further!

Almost any scenario that you believe requires a CMS is a candidate for a headless CMS too. However, headless CMSs also have uses beyond their traditional content serving functions. If you look at the tech trends of the last couple of years, you might have realised that chatbots, smart devices and digital assistants are topping the charts. These interfaces are powered by the innovation in hardware, NUI (natural user interface) and of course, the content that can be served in a variety of ways.

A good chatbot is powered by state-of-art NLP models, and optionally, with a voice interface that allows it

to understand what is being asked in natural language either using text or voice inputs. Once the processing for this task is over, the bot makes itself useful by surfacing the relevant content. This is similar for the digital assistants — in their hardware as well as software avatars.

The content that you consume this way is usually small snippets of information that can be sourced from emails, calendars, third party services such as flight tracking and weather, etc. But other content, such as the witty responses that you get, are best served from a headless content. These aren't only limited to jokes, etc, but include meaningful content as well.

As an example, consider a food ordering or a shopping portal. These services take a large number of partners and sellers onto their platform, who list their services or products. A traditional approach is to store this content in a NoSQL DB and fetch the content as the UI is being rendered. On a high level, this approach requires that you build an API wrapper that does the following:

- Builds a UI to take onboard the partners, sellers, items and services
- Validates the content before it is saved in the database with the right structure
- Fetches the content when being served to the users

Add a chatbot or a digital assistant (or a skill) to the mix and you can serve the content on those mediums too if your APIs are designed efficiently.

With a headless CMS, you are left with just the work needed for Point 1

listed above, as a headless CMS provides first class APIs to interact with the data and data types so you don't have to worry about database handling. You can get started with how you will integrate these APIs with your application surfaces, be it a mobile or Web app, chatbot or a skill for a digital assistant.

Strapi (<https://strapi.io/>) is one of the headless CMSs that I am working with, and it includes a UI to add content as well as user roles and permissions to control access to the content. Ghost (<https://ghost.org/>) and Netlify CMS (<https://www.netlifycms.org/>) are others that I have tried out a little bit. You can look at other options at <https://headlesscms.org/>.

A headless CMS also makes an excellent case for Web apps or websites, even the ones that are a good mix of static and dynamic content. You can front-end your headless CMS with a static site generator such as Hugo or Gatsby, and achieve much greater performance while lowering your computing costs. Static site generators also support plugins to fetch data from dynamic sources such as APIs. This combination offers a number of benefits when used in conjunction with a CDN. Dynamic site acceleration solutions such as Azure Front Door or CloudFront add an additional layer of performance benefits that can further reduce the load on your content repository. A prime example of this scenario is GitHub Pages (<https://pages.github.com/>) that allows you to host a static website directly from your GitHub repo, free of cost. Figure 2 is a schematic for this scenario.

Variations of this architecture can be useful in other scenarios where serving content is part of the solution.

Challenges with a headless CMS

When looking at the reference architecture diagram, it is evident that the lack of a presentation layer in a headless CMS introduces the flexibility to use any front-end. But it also means that you have another piece of the puzzle that



Figure 2: A schematic for GitHub Pages